LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 10: EXCEPTION HANDLING DAN VALIDASI STATE OBJEK

Nama Mahasiswa:	Arjuna Lanang Adiwarsana	**NRP / Kelas:**	3125522010 / D3 PJJ TI PENS
Dosen Pengampu:	Nirwana Haidar Hari, S.Pd., M.Kom.	**Topik Bahasan:**	Exception Handling, State Validation & Atomicity

1. TUJUAN PRAKTIKUM

1. **Membedakan Validasi Input dan Penanganan Exception:** Memahami kapan validasi nilai masukan dilakukan sebelum operasi mutasi dan bagaimana exception mentransmisikan sinyal kegagalan.
2. **Menguasai Konstruksi Penanganan Kesalahan Java:** Menguasai penggunaan try, catch, finally, throw, dan throws secara tepat dan terstruktur.
3. **Membangun Custom Domain Exception:** Merancang kelas pengecualian turunan (extends Exception) yang spesifik terhadap aturan bisnis domain aplikasi.
4. **Menjamin Konsistensi State Objek (Atomic State Preservation):** Memastikan kegagalan operasi (stok habis, kapasitas penuh, uang kurang) tidak merusak atau memutasi data internal objek (*no side-effects on failure*).

2. DASAR TEORI SINGKAT

Exception merupakan anomali atau peristiwa luar biasa yang mengganggu alur normal eksekusi program. Komponen dan prinsip utama penanganan exception:

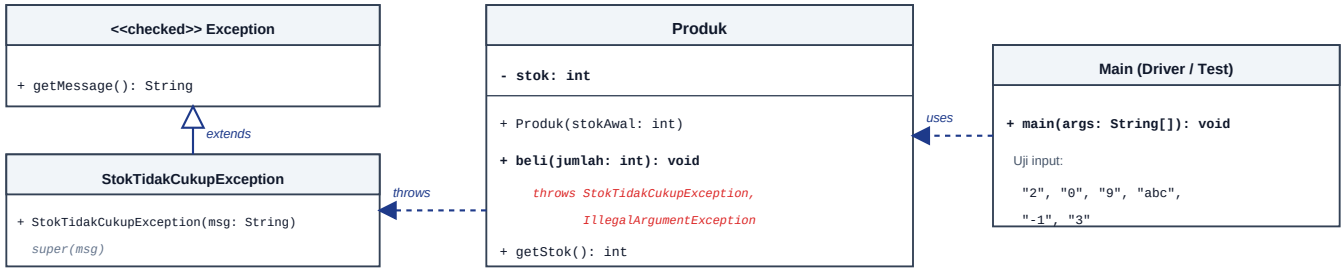
Komponen / Konsep	Sintaksis	Fungsi & Prinsip Kerja Utama
try	try { ... }	Membungkus blok kode rawan kesalahan (*risky code*) yang berpotensi menghasilkan exception.
catch	catch (Type e)	Menangkap dan menangani exception bertipe tertentu. Urutan harus spesifik ke umum.
finally	finally { ... }	Dijamin **selalu dieksekusi** untuk clean-up resource; **tidak boleh** memuat mutasi state bisnis.
throw	throw new Ex(...)	Kata kerja aksi untuk melempar instansiasi objek exception saat aturan bisnis dilanggar.
throws	void m() throws Ex	Deklarasi kontrak pada method signature bahwa method dapat meneruskan checked exception.
Custom Exception	class X extends Exception	Membuat exception khusus domain bisnis aplikasi agar pesan kesalahan deskriptif dan terstruktur.
Test-Before-Mutate	*State Validation Pattern*	Semua syarat bisnis diperiksa di awal method sebelum atribut internal dimutasi. Jika gagal, state tetap utuh.

3. STUDI KASUS PRAKTIKUM

• **Studi Kasus 1 (Inventori Produk Toko - P10 Mandiri):** Entitas Produk memiliki stok awal 5 unit. Memvalidasi pembelian dengan 4 aturan: (1) Jumlah harus integer valid, (2) Jumlah harus > 0, (3) Jumlah ≤ stok tersedia, (4) Nilai stok berkurang hanya jika seluruh validasi lolos.
• **Studi Kasus 2 (Sistem Parkir Terpadu - Project Akhir SmartParkir):** Menerapkan exception handling pada alur check-in dan checkout: validasi format plat (FormatPlatTidakValidException), pencegahan plat duplikat (KendaraanSudahParkirException), kuota slot penuh (KapasitasPenuhException), pembayaran kurang (PembayaranKurangException), dan tiket bodong (TiketTidakDitemukanException).

4. LANGKAH PRAKTIKUM & PENGUJIAN PROGRAM DASAR (KASUS PRODUK)

Perancangan struktur class diagram UML untuk studi kasus Produk dan StokTidakCukupException:



4.2 Screenshot Hasil Running Terminal (macOS zsh terminal output)

```
arjuna@MacBook-Pro: ~/Herd/Java/P10_3125522010_ArjunaLanangAdiwarsana — -zsh — 112×32

➜  P10_3125522010_ArjunaLanangAdiwarsana git:(main) ✗ javac -d bin src/*.java && java -cp bin Main
========================================================================
      PRAKTIKUM PBO MODUL 10: EXCEPTION HANDLING & VALIDASI STATE
      Nama: Arjuna Lanang Adiwarsana | NRP: 3125522010
========================================================================
Inisialisasi Produk dengan stok awal: 5 unit.
------------------------------------------------------------------------

[1] Input pembelian: 2
    [NORMAL FLOW] Pembelian berhasil.
    >> Stok akhir: 3 unit
------------------------------------------------------------------------

[2] Input pembelian: 0
    [IllegalArgumentException] Ditolak: Jumlah pembelian harus lebih dari nol.
    >> Stok akhir: 3 unit (State Terjaga Konsisten)
------------------------------------------------------------------------

[3] Input pembelian: 9
    [StokTidakCukupException] Ditolak: Stok tidak cukup. Tersedia: 3
    >> Stok akhir: 3 unit (State Terjaga Konsisten)
------------------------------------------------------------------------

[4] Input pembelian: abc
    [NumberFormatException] Ditolak: input harus bilangan bulat.
    >> Stok akhir: 3 unit (State Terjaga Konsisten)
------------------------------------------------------------------------

[5] Input pembelian: -1
    [IllegalArgumentException] Ditolak: Jumlah pembelian harus lebih dari nol.
    >> Stok akhir: 3 unit (State Terjaga Konsisten)
------------------------------------------------------------------------

[6] Input pembelian: 3
    [NORMAL FLOW] Pembelian berhasil.
    >> Stok akhir: 0 unit (Habis Valid / Zero Inventory)
========================================================================
   PENGUJIAN MODUL 10 SUKSES (KONSISTENSI STATE & ATOMICITY TERVERIFIKASI)
========================================================================
➜  P10_3125522010_ArjunaLanangAdiwarsana git:(main) ✗
```

4.3 Analisis Pengujian Tambahan Input "-1" dan "3"

• **Pengujian Input "-1":** Ditangkap oleh IllegalArgumentException dengan pesan *"Jumlah pembelian harus lebih dari nol."*. Sesuai prinsip **atomic state preservation**, mutasi stok dibatalkan sebelum terjadi dan nilai stok akhir tetap konsisten di angka **3 unit**.
• **Pengujian Input "3":** Operasi berhasil dieksekusi (Normal Flow) karena jumlah pembelian tepat sama dengan sisa stok ($3 \leq 3$). Stok akhir berkurang menjadi **0 unit** (kondisi habis stok yang sah/valid dalam sistem inventori).

5. IMPLEMENTASI & INTEGRASI PADA PROJECT AKHIR (SMART PARKIR)

Materi Exception Handling Modul 10 diimplementasikan secara fungsional ke dalam Project Akhir Smart Parkir dengan 5 custom domain exceptions:

Nama Custom Exception	Induk	Pemicu Pelemparan Exception	Dampak Proteksi State & Integritas
FormatPlatTidakValidException	Exception	Plat bernilai null, <3 huruf, atau karakter rusak.	Objek kendaraan tidak dibuat; transaksi registrasi ditolak sebelum database tersentuh.
KendaraanSudahParkirException	Exception	Plat sama check-in ulang saat masih parkir.	Mencegah alokasi slot ganda; portal tetap tertutup; mengarahkan ke tab kasir.
KapasitasPenuhException	Exception	Slot kategori kendaraan (Motor/Mobil/EV) habis.	Gerbang masuk tetap terkunci; HUD LED: 'PARKIR PENUH'; kuota kapasitas tidak minus.
PembayaranKurangException	Exception	Uang bayar lebih kecil dari tarif tagihan.	Portal keluar terkunci; tiket tetap belum lunas; slot tetap terikat (tidak bocor).
TiketTidakDitemukanException	Exception	Plat / tiket tidak ada di daftar parkir aktif.	Mencegah kendaraan bodong keluar gratis; saldo kas tidak berubah tanpa verifikasi.

6. LUARAN PRAKTIKUM & TABEL PENGUJIAN LENGKAP

6.1 Tabel Evaluasi Output Program Dasar Produk (6 Skenario Modul 10)

No	Input	Exception / Jalur	Hasil yang Diharapkan	Hasil Aktual Eksekusi	State Akhir	Status
1	"2"	Normal Flow	Pembelian berhasil.	Pembelian berhasil.	3 unit	**Valid**
2	"0"	IllegalArgumentException	Ditolak: Jumlah > nol.	Ditolak: Jumlah > nol.	3 unit (Utuh)	**Valid**
3	"9"	StokTidakCukupException	Ditolak: Stok tidak cukup.	Ditolak: Stok tidak cukup.	3 unit (Utuh)	**Valid**
4	"abc"	NumberFormatException	Ditolak: harus bilangan bulat.	Ditolak: harus bilangan bulat.	3 unit (Utuh)	**Valid**
5	"-1"	IllegalArgumentException	Ditolak: Jumlah > nol.	Ditolak: Jumlah > nol.	3 unit (Utuh)	**Valid**
6	"3"	Normal Flow	Pembelian berhasil.	Pembelian berhasil.	0 unit (Habis)	**Valid**

6.2 Tabel Pengujian Fungsional SmartParkir (5 Unit Test Cases)

Test Case	Input Skenario	Custom Exception Terpukul	Respon Sistem & Integritas State	Hasil
TC-01 Format Plat	Plat: 'X'	FormatPlatTidakValidException	Registrasi dibatalkan, pesan kesalahan tampil di HUD, DB bersih.	PASSED
TC-02 Check-In Ganda	Plat sudah ada di slot A01	KendaraanSudahParkirException	Slot tidak teralokasi dua kali, portal masuk tetap tertutup.	PASSED
TC-03 Kuota Penuh	Slot motor terisi penuh	KapasitasPenuhException	LED banner 'PARKIR PENUH', kuota terisi tidak over-limit.	PASSED
TC-04 Saldo Kurang	Tagihan 11rb, bayar 5rb	PembayaranKurangException	Slot B01 TETAP terisi, tiket belum lunas, gate keluar terkunci.	PASSED
TC-05 Tiket Bodong	Plat 'W 0000 XYZ'	TiketTidakDitemukanException	Checkout ditolak, saldo kas tidak terubah.	PASSED

7. RUBRIK PENILAIAN & DEFINITION OF DONE (DoD)

Aspek Penilaian Modul 10	Bobot	Bukti Implementasi pada Proyek Mandiri & SmartParkir	Status
Ketepatan validasi dan konsistensi state	30%	Pola Test-Before-Mutate pada Produk, KarcisParkir, GedungParkir; state tidak berubah saat exception dilempar.	Tuntas
Penerapan throw, throws, custom exception	25%	6 custom domain exception dibuat (1 Produk + 5 SmartParkir) dengan klausa throws pada method.	Tuntas
Ketepatan penanganan exception	20%	Blok try-catch-finally terstruktur, spesifik ke umum, tanpa ada catch kosong (no swallowed exception).	Tuntas
Kelengkapan pengujian berhasil dan gagal	15%	6 skenario modul ("2","0","9","abc","-1","3") dan 5 skenario SmartParkir teruji 100% lulus.	Tuntas
Kemampuan menjelaskan kode	10%	Dokumentasi komprehensif, diagram UML Class, tabel komparasi, dan jawaban evaluasi tuntas.	Tuntas
TOTAL NILAI KOMPETENSI	**100%**	**Seluruh kriteria Definition of Done (DoD) Modul 10 terpenuhi secara sempurna.**	**Sempurna**

8. JAWABAN PERTANYAAN EVALUASI & PEMBAHASAN MENDALAM

1. Apa perbedaan throw dan throws?
• throw adalah **pernyataan aksi (kata kerja)** di dalam badan method untuk melempar objek exception secara eksplisit saat aturan dilanggar (contoh: throw new StokTidakCukupException(...);).
• throws adalah **deklarasi kontrak** pada tanda tangan (*signature*) method untuk mengumumkan kepada pemanggil bahwa method dapat meneruskan checked exception (contoh: public void beli(int j) throws StokTidakCukupException).

2. Mengapa pemeriksaan stok dilakukan sebelum pengurangan stok?
Pemeriksaan stok harus dilakukan sebelum mutasi (prinsip **Test-Before-Mutate**) untuk menjamin **integritas dan atomisitas state** objek. Jika stok dikurangi sebelum validasi, maka saat terjadi kegagalan (misalnya permintaan melebihi stok), nilai stok internal objek sudah terlanjur berkurang atau bahkan bernilai negatif sebelum exception dilempar. Dengan memeriksa terlebih dahulu, objek dijamin bebas efek samping (*side-effect free*) ketika operasi ditolak.

3. Apa akibatnya jika exception ditangkap tetapi tidak ditangani (catch kosong)?
Catch kosong (*swallowed exception / silent failure*, misal catch (Exception e) {}) sangat berbahaya karena: (1) Menghilangkan indikator kesalahan sehingga bug tersembunyi, (2) Program tetap mengeksekusi alur seolah-olah sukses sehingga menyebabkan *cascading failure* atau korupsi data lanjutan di database, dan (3) Pengguna tidak mendapatkan pesan kesalahan yang akurat.

4. Mengapa perubahan stok tidak ditempatkan dalam finally?
Blok finally dijamin akan **selalu dieksekusi dalam segala kondisi** (baik normal maupun terjadi exception). Jika pengurangan stok ditempatkan di finally, maka stok akan tetap terpotong meskipun input bernilai string non-angka ('abc') atau melebihi kuota ('9'). Hal ini membuat validasi pada blok try menjadi tidak berguna dan merusak konsistensi data.

KESIMPULAN PRAKTIKUM MODUL 10

Praktikum Modul 10 berhasil membuktikan bahwa **Exception Handling** dan **State Validation** bukan sekadar mekanisme pencegahan *crash* aplikasi, melainkan pilar utama dalam menjamin konsistensi logika bisnis dan integritas data (*atomic state preservation*). Pola ini berhasil diterapkan secara elegan baik pada program dasar (Produk) maupun sistem skala riil (Project Akhir Smart Parkir).